2a. Univariate Analysis: Frequency, Mean, Median, Mode, Variance, Standard Deviation, Skewness and Kurtosis

```
df.mean()
```

```
Pregnancies                   3.845052
Glucose                     120.894531
BloodPressure                69.105469
SkinThickness                20.536458
Insulin                      79.799479
BMI                          31.992578
DiabetesPedigreeFunction      0.471876
Age                          33.240885
Outcome                       0.348958
dtype: float64
```

```
print(df.loc[:,'Glucose'].mean())
print(df.loc[:,'Age'].mean())
```

```
120.89453125
33.240885416666664
```

```
df.median()
```

```
Pregnancies                   3.0000
Glucose                     117.0000
BloodPressure                72.0000
SkinThickness                23.0000
Insulin                      30.5000
BMI                          32.0000
DiabetesPedigreeFunction      0.3725
Age                          29.0000
Outcome                       0.0000
dtype: float64
```

```
# Calculate the standard deviation of the first five rows
df.std(axis=1)[0:5]
```

```
0    48.296112
1    31.119744
2    59.585320
3    37.639873
4    60.541569
dtype: float64
```

```
# Calculate Variance
df.var()
```

```
Pregnancies                   11.354056
Glucose                     1022.248314
BloodPressure                374.647271
SkinThickness                254.473245
Insulin                    13281.180078
BMI                           62.159984
DiabetesPedigreeFunction       0.109779
Age                          138.303046
Outcome                        0.227483
dtype: float64
```

```
# Calculate Skewness
print(df.skew())
```

```
Pregnancies                 0.901674
Glucose                     0.173754
BloodPressure              -1.843608
SkinThickness               0.109372
Insulin                     2.272251
BMI                        -0.428982
DiabetesPedigreeFunction    1.919911
Age                         1.129597
Outcome                     0.635017
dtype: float64
```

```
# To calculate a median of a particular column
print(df.loc[:,'Age'].median())
print(df.loc[:,'BMI'].median())
```

```
29.0
32.0
```

```
#To calculate the mode
df.mode()
```

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	DiabetesPedigreeFunction	Age	Outcome
0	1.0	99	70.0	0.0	0.0	32.0	0.254	22.0	0.0
1	NaN	100	NaN	NaN	NaN	NaN	0.258	NaN	NaN

```
# To calculate Standard Deviation
df.std()
```

```
Pregnancies                   3.369578
Glucose                      31.972618
BloodPressure                19.355807
SkinThickness                15.952218
Insulin                     115.244002
BMI                           7.884160
DiabetesPedigreeFunction      0.331329
Age                          11.760232
Outcome                       0.476951
dtype: float64
```

```
print(df.loc[:,'Age'].std())
```

```
11.76023154067868
```

```
# Calculate skewness
skew(df,bias=False)
```

```
array([ 0.90167398,  0.1737535 , -1.84360798,  0.1093725 ,  2.27225086,
       -0.42898159,  1.91991107,  1.1295967 ,  0.63501664])
```

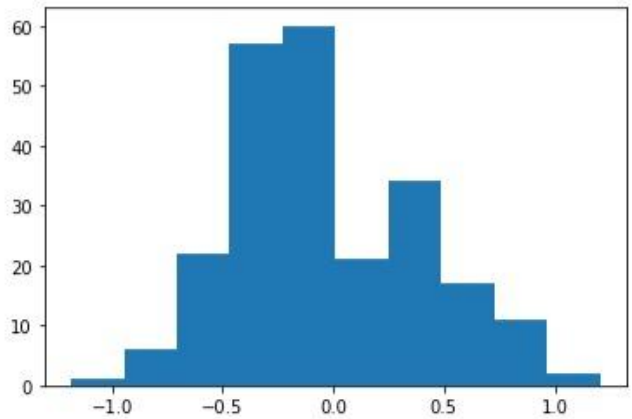
```
# Calculate kurtosis
kurtosis(df, bias=False)
```

```
array([ 0.15921978,  0.64077982,  5.18015656, -0.52007187,  7.21425955,
        3.2904429 ,  5.59495353,  0.64315889, -1.60092976])
```

2b..Bivariate Analysis

```
plt.hist(y_test - predictions)
```

```
(array([ 1.,  6., 22., 57., 60., 21., 34., 17., 11.,  2.]),
 array([-1.18545987, -0.94652527, -0.70759067, -0.46865606, -0.22972146,
        0.00921314,  0.24814774,  0.48708234,  0.72601695,  0.96495155,
        1.20388615]),
 <BarContainer object of 10 artists>)
```




```
print(model_score)
print(metrics.confusion_matrix(y_test,y_predict))
```

```
0.7489177489177489
[[127  20]
 [ 38  46]]
```

```
data.head().transpose()
```

	0	1	2	3	4
Pregnancies	6.000	1.000	8.000	1.000	0.000
Glucose	148.000	85.000	183.000	89.000	137.000
BloodPressure	72.000	66.000	64.000	66.000	40.000
SkinThickness	35.000	29.000	0.000	23.000	35.000
Insulin	0.000	0.000	0.000	94.000	168.000
BMI	33.600	26.600	23.300	28.100	43.100
DiabetesPedigreeFunction	0.627	0.351	0.672	0.167	2.288
Age	50.000	31.000	32.000	21.000	33.000
Outcome	1.000	0.000	1.000	0.000	1.000

2C.Multiple regression analysis

```
import pandas
from sklearn import linear_model

df = pandas.read_csv("diabetes.csv")

X = df[['Glucose', 'BloodPressure']]
y = df['Outcome']

regr = linear_model.LinearRegression()
regr.fit(X, y)

#predict the Outcome when Glucose is 175 and BloodPressure is 92
predictedOutcome = regr.predict([[75, 72]])

print(predictedOutcome)
```

```
[0.02841926]
```

3. Apply Bayesian and SVM techniques on Iris and Diabetes data set.

```
X = dataset.iloc[:,:4].values
y = dataset['Species'].values
dataset.head(5)
```

	Unnamed: 0	Sepal.Length	Sepal.Width	Petal.Length	Petal.Width	Species
0	51	7.0	3.2	4.7	1.4	versicolor
1	52	6.4	3.2	4.5	1.5	versicolor
2	53	6.9	3.1	4.9	1.5	versicolor
3	54	5.5	2.3	4.0	1.3	versicolor
4	55	6.5	2.8	4.6	1.5	versicolor

```
#Step 5: Training the Naive Bayes Classification model on the Training Set
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
classifier.fit(X_train, y_train)
```

```
GaussianNB()
```

```
#Step 6: Predicting the Test set results
y_pred = classifier.predict(X_test)
y_pred
```

```
array(['virginica', 'virginica', 'virginica', 'virginica', 'versicolor',
       'virginica', 'virginica', 'versicolor', 'versicolor', 'versicolor',
       'virginica', 'versicolor', 'virginica', 'virginica', 'virginica',
       'versicolor', 'virginica', 'virginica', 'versicolor', 'versicolor'],
      dtype='<U10')
```

```
#Step 8: Comparing the Real Values with Predicted Values
df = pd.DataFrame({'Real Values':y_test, 'Predicted Values':y_pred})
df
```

	Real Values	Predicted Values
0	virginica	virginica
1	virginica	virginica
2	virginica	virginica
3	virginica	virginica
4	versicolor	versicolor
5	virginica	virginica

```
# Let's see a sample of created df
df.sample(frac=0.01)
```

	sepal length (cm)	sepal width (cm)	petal length (cm)	petal width (cm)	target
90	5.5	2.6	4.4	1.2	1
19	5.1	3.8	1.5	0.3	0

```
# Let's see target names
targets = iris.target_names
print(targets)
```

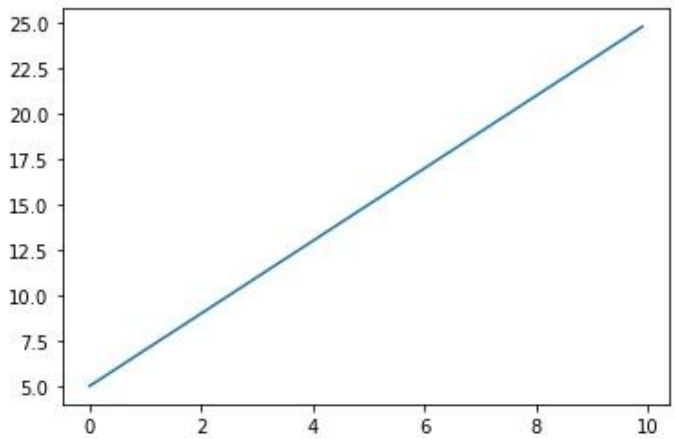
```
['setosa' 'versicolor' 'virginica']
```

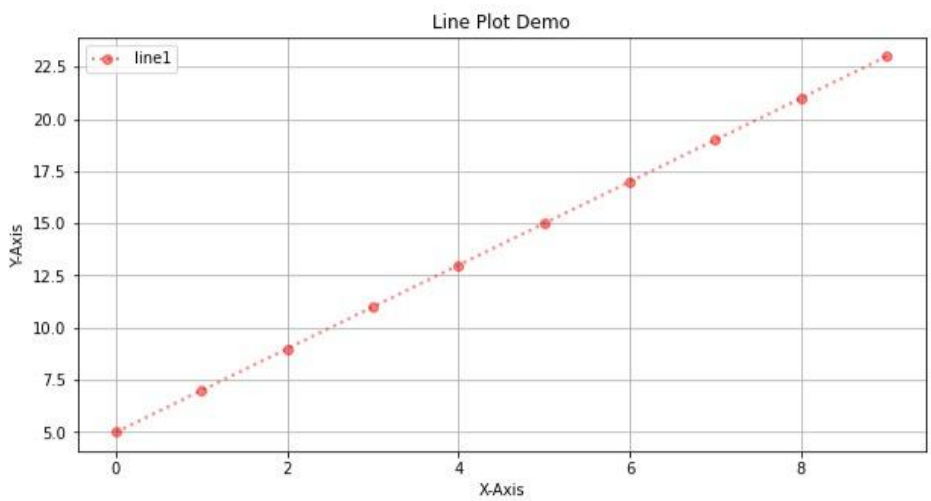
```
print("Prediction is: {}".format(targets[y_pred]))
```

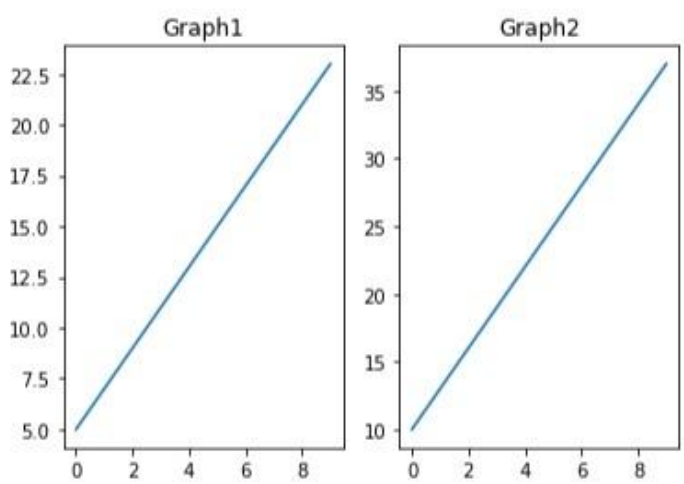
```
Prediction is: ['setosa']
```

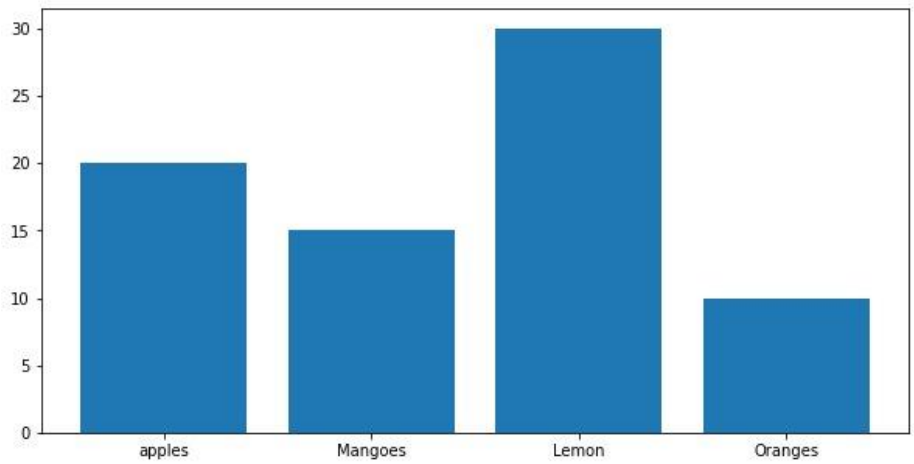
4. Apply and explore various plotting functions on UCI data sets

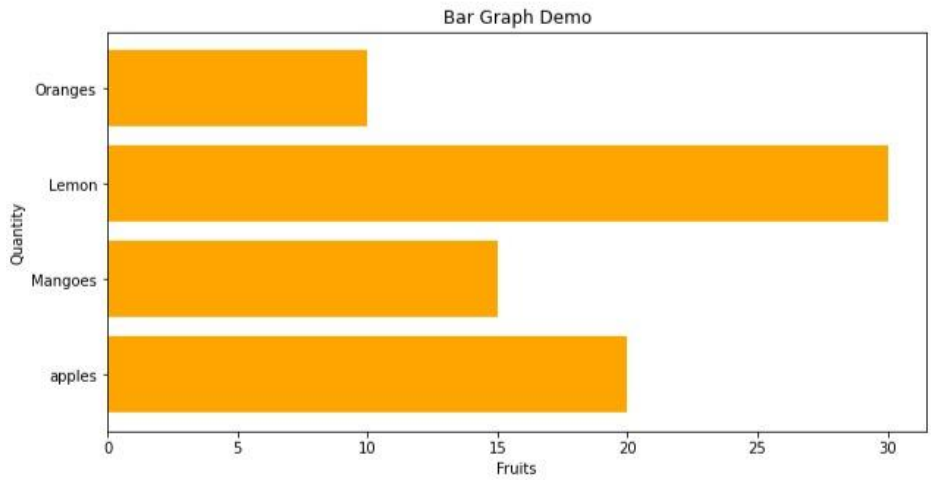
```
#plotting the datapoints
plt.plot(x,y)
plt.show()
```

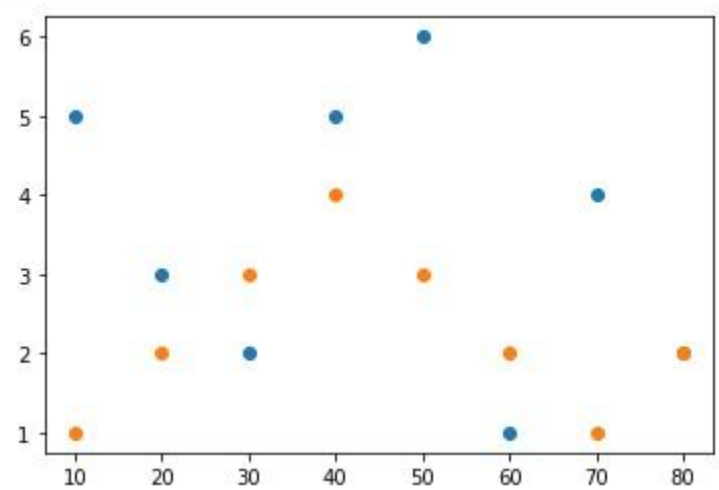


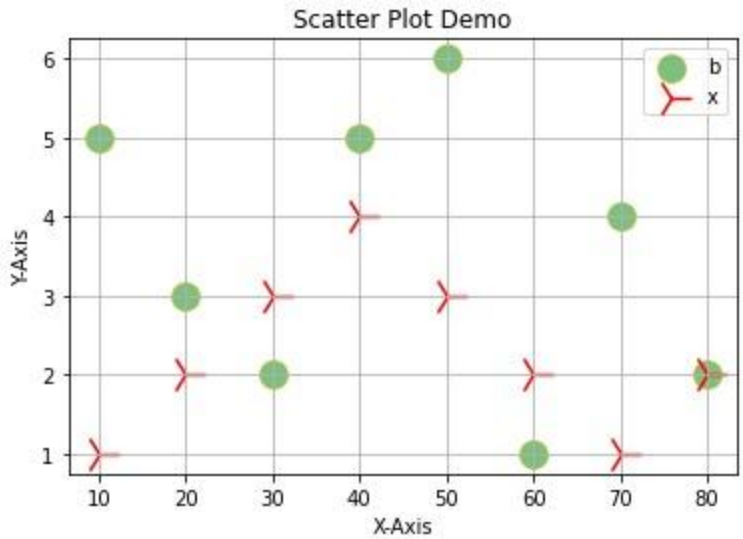


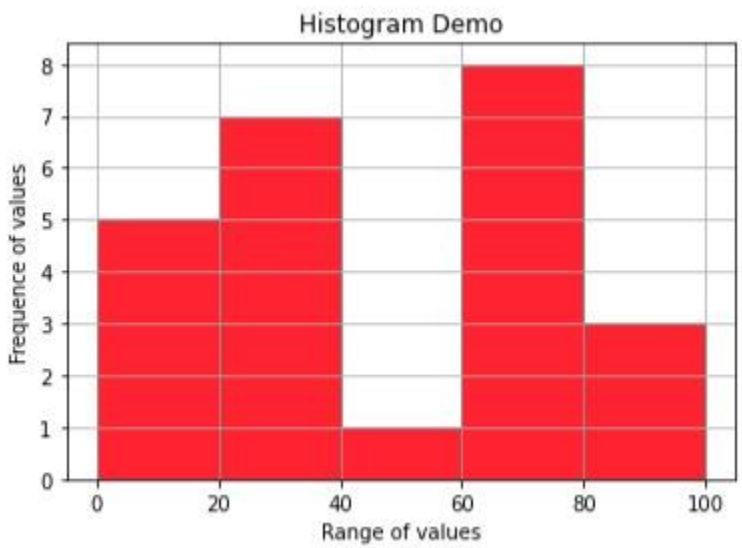


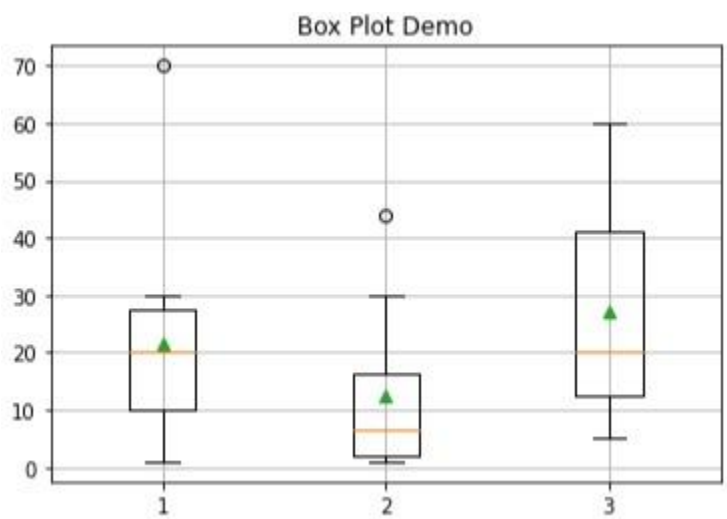

30
25
20
15
10
5
0
apples
Mangoes
Lemon
Oranges

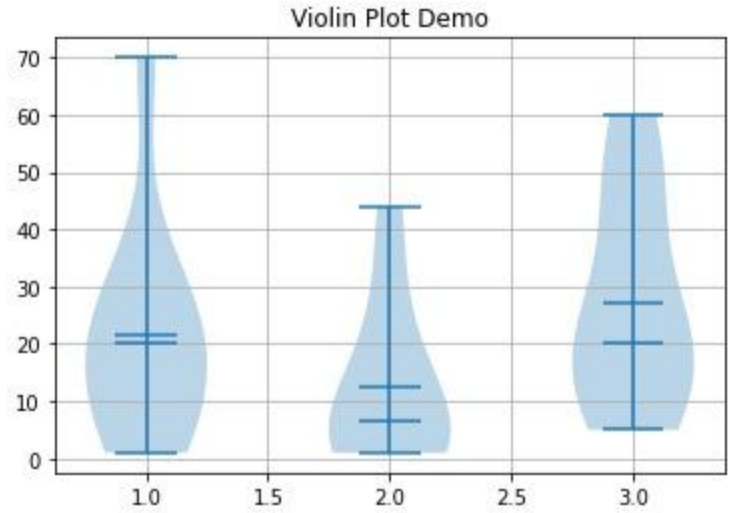

Bar Graph Demo
Oranges
Lemon
Mangoes
apples
Quantity
0
5
10
15
20
25
30
Fruits

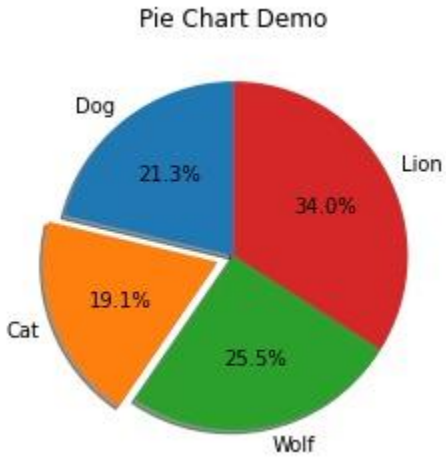

6
5
4
3
2
1
10
20
30
40
50
60
70
80

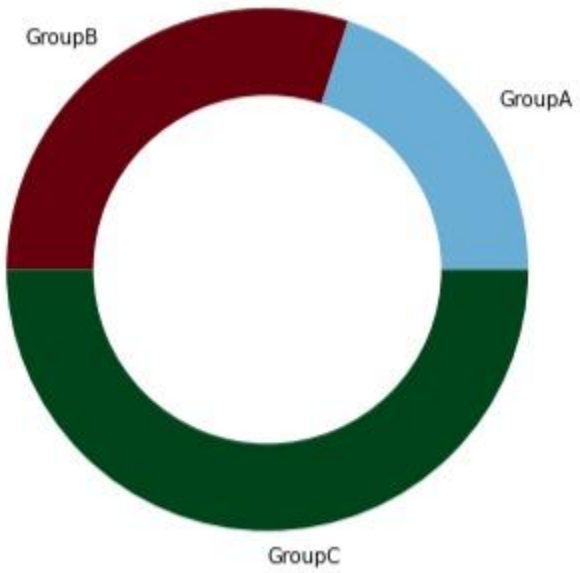

Scatter Plot Demo
b
x
Y-Axis
X-Axis
1
2
3
4
5
6
10
20
30
40
50
60
70
80

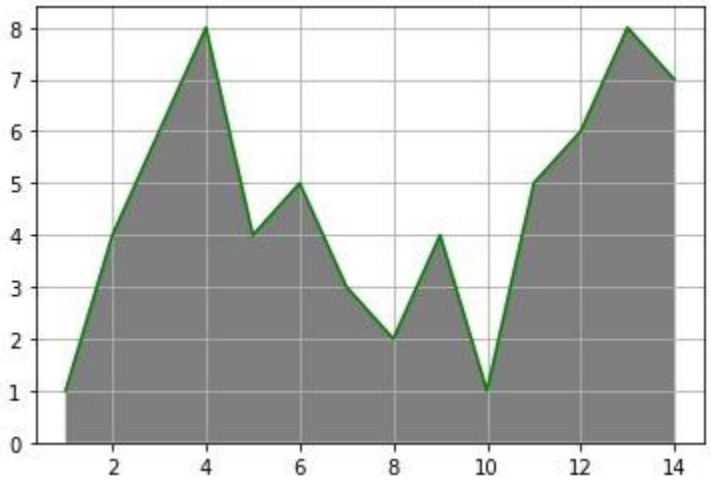

Histogram Demo
Frequence of values
Range of values
0
1
2
3
4
5
6
7
8
0
20
40
60
80
100


Box Plot Demo
0
10
20
30
40
50
60
70
1
2
3


Violin Plot Demo
70
60
50
40
30
20
10
0
1.0
1.5
2.0
2.5
3.0


Pie Chart Demo
Dog
21.3%
Lion
34.0%
19.1%
Cat
25.5%
Wolf


GroupB
GroupA
GroupC


8
7
6
5
4
3
2
1
0
2
4
6
8
10
12
14